rqbit API 타입 및 직렬화 분석

개요

이 문서는 rqbit BitTorrent 클라이언트의 HTTP API에서 사용하는 요청/응답 타입들과 JSON 직렬화/역직렬화 구현을 분석합니다. rqbit은 Serde를 사용하여 강력한 타입 안전성을 제공하면서도 유연한 API 인터페이스를 구현하고 있습니다.

요청 타입 분석

1. 토렌트 추가 요청 파라미터

TorrentAddQueryParams 구조체

```
#[derive(Serialize, Deserialize, Default)]
pub struct TorrentAddQueryParams {
pub overwrite: Option<bool>,
pub output_folder: Option<String>,
pub sub_folder: Option<String>,
pub only_files_regex: Option<String>,
pub only_files: Option<OnlyFiles>,
pub peer_connect_timeout: Option<u64>,
pub peer_read_write_timeout: Option<u64>,
pub initial_peers: Option<InitialPeers>,
```

```
pub is_url: Option<bool>,

pub list_only: Option<bool>,

}
```

이 구조체는 토렌트 추가 시 사용되는 모든 옵션들을 포함합니다:

- **overwrite**: 기존 파일 덮어쓰기 여부
- **output_folder**: 다운로드 대상 폴더
- **sub_folder**: 하위 폴더 생성
- **only_files_regex**: 정규식을 통한 파일 필터링
- **only_files**: 특정 파일 인덱스만 다운로드
- **peer_connect_timeout**: 피어 연결 타임아웃
- **peer_read_write_timeout**: 피어 읽기/쓰기 타임아웃
- **initial_peers**: 초기 피어 목록
- **is_url**: 콘텐츠가 URL인지 강제 지정
- **list_only**: 메타데이터만 가져오기

커스텀 직렬화 타입들

OnlyFiles 타입

```
pub struct OnlyFiles(Vec<usize>);


impl Serialize for OnlyFiles {

fn serialize<S>(&self, serializer: S) -> core::result::Result<S::Ok, S::Error>

where

S: serde::Serializer,
```

```
{
let s = self.0.iter().map(|id| id.to_string()).join(",");
s.serialize(serializer)
}
}


impl<'de> Deserialize<'de> for OnlyFiles {
fn deserialize<D>(deserializer: D) -> core::result::Result<Self, D::Error>
where
D: serde::Deserializer<'de>,
{
let s = String::deserialize(deserializer)?;
let list = s.split(',')
.try_fold(Vec::<usize>::new(), |mut acc, c| match c.parse() {
Ok(i) => { acc.push(i); Ok(acc) }
Err(_) => Err(D::Error::custom(format!(
"only_files: failed to parse {:?} as integer", c
))),
})?;
if list.is_empty() {
return Err(D::Error::custom(
"only_files: should contain at least one file id",
));
```

```
}
Ok(OnlyFiles(list))
}
}
```

특징:

- **쉼표 구분 문자열**: `"1,2,3"` 형태로 직렬화
- **파싱 검증**: 각 요소가 유효한 정수인지 확인
- **빈 목록 방지**: 최소 하나의 파일 ID 필요
- **상세한 오류 메시지**: 파싱 실패 시 구체적인 오류 정보 제공

InitialPeers 타입

```
pub struct InitialPeers(pub Vec<SocketAddr>);

impl<'de> Deserialize<'de> for InitialPeers {
fn deserialize<D>(deserializer: D) -> std::prelude::v1::Result<Self, D::Error>
where
D: serde::Deserializer<'de>,
{
let string = String::deserialize(deserializer)?;
let mut addrs = Vec::new();
for addr_str in string.split(',').filter(|s| !s.is_empty()) {
addrs.push(SocketAddr::from_str(addr_str).map_err(D::Error::custom)?);
```

```
}
Ok(InitialPeers(addrs))
}
}


impl Serialize for InitialPeers {
fn serialize<S>(&self, serializer: S) -> std::prelude::v1::Result<S::Ok, S::Erro
where
S: serde::Serializer,
{
self.0.iter()
.map(|s| s.to_string())
.join(",")
.serialize(serializer)
}
}
```

특징:

- **네트워크 주소 파싱**: `"192.168.1.1:8080,10.0.0.1:6881"` 형태
- **빈 문자열 필터링**: 공백 요소 자동 제거
- **SocketAddr 검증**: 유효한 IP:포트 형식 확인

2. 토렌트 식별자 타입

TorrentIdOrHash 열거형

```rust
#[derive(Debug, Clone, Copy)]
pub enum TorrentIdOrHash {
Id(TorrentId),
Hash(Id20),
}
```

이 타입은 토렌트를 식별하는 두 가지 방법을 지원합니다:

- **Id**: 내부 토렌트 ID (정수)
- **Hash**: SHA-1 해시 (40자 16진수 문자열)

유연한 역직렬화 구현

```rust
impl<'de> Deserialize<'de> for TorrentIdOrHash {
fn deserialize<D>(deserializer: D) -> std::result::Result<Self, D::Error> {
impl<'de> serde::de::Visitor<'de> for V<'de> {
type Value = TorrentIdOrHash;

fn expecting(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
f.write_str("integer or 40 byte info hash")
}

fn visit_i64<E>(self, v: i64) -> std::result::Result<Self::Value, E> {
let tid: TorrentId = v.try_into().map_err(|e| E::custom(format!("{e:#}")))?;
Ok(TorrentIdOrHash::from(tid))
```

```
}

fn visit_str<E>(self, v: &str) -> std::result::Result<Self::Value, E> {
TorrentIdOrHash::parse(v).map_err(|e| {
E::custom(format!(
"expected integer or 40 byte info hash, couldn't parse string: {e:#}"
))
})
}
}
deserializer.deserialize_any(V::default())
}
}
```

특징:

- **다형성 지원**: 정수와 문자열 모두 처리
- **자동 타입 감지**: 40자 문자열은 해시로, 그 외는 ID로 처리
- **명확한 오류 메시지**: 파싱 실패 시 기대값과 실제값 표시

3. 쿼리 파라미터 옵션들

ApiTorrentListOpts

```
#[derive(Deserialize, Default)]
pub struct ApiTorrentListOpts {
```

```
#[serde(default)]
pub with_stats: bool,
}
```

토렌트 목록 조회 시 통계 정보 포함 여부를 제어합니다.

응답 타입 분석

1. 토렌트 목록 응답

TorrentListResponse

```
#[derive(Serialize)]
pub struct TorrentListResponse {
pub torrents: Vec<TorrentDetailsResponse>,
}
```

TorrentDetailsResponse

```
#[derive(Serialize, Deserialize)]
pub struct TorrentDetailsResponse {
#[serde(skip_serializing_if = "Option::is_none")]
pub id: Option<usize>,
pub info_hash: String,
pub name: Option<String>,
```

```
pub output_folder: String,


#[serde(skip_serializing_if = "Option::is_none")]
pub files: Option<Vec<TorrentDetailsResponseFile>>,
#[serde(skip_serializing_if = "Option::is_none", skip_deserializing)]
pub stats: Option<TorrentStats>,
}
```

특징:

- **조건부 직렬화**: `skip_serializing_if` 를 사용하여 None 값 제외
- **읽기 전용 필드**: `stats` 는 역직렬화에서 제외
- **유연한 구조**: 상황에 따라 다른 필드 조합 제공

TorrentDetailsResponseFile

```
#[derive(Serialize, Deserialize)]
pub struct TorrentDetailsResponseFile {
pub name: String,
pub components: Vec<String>,
pub length: u64,
pub included: bool,
pub attributes: FileDetailsAttrs,
}
```

각 파일의 상세 정보를 포함:

- **name**: 파일명
- **components**: 경로 구성 요소들
- **length**: 파일 크기
- **included**: 다운로드 대상 여부
- **attributes**: 파일 속성 (실행 가능, 심볼릭 링크 등)

2. 토렌트 추가 응답

ApiAddTorrentResponse

```
#[derive(Serialize, Deserialize)]
pub struct ApiAddTorrentResponse {
pub id: Option<usize>,
pub details: TorrentDetailsResponse,
pub output_folder: String,
pub seen_peers: Option<Vec<SocketAddr>>,
}
```

토렌트 추가 결과와 함께 발견된 피어 정보도 제공합니다.

3. 통계 응답 타입들

SessionStatsSnapshot

```
#[derive(Debug, Serialize)]
pub struct SessionStatsSnapshot {
pub counters: SessionCountersSnapshot,
```

```
pub download_speed: Speed,
pub upload_speed: Speed,
pub peers: AggregatePeerStats,
pub uptime_seconds: u64,
pub connections: ConnectStatsSnapshot,
}
```

전역 세션 통계를 제공하며, Prometheus 메트릭 형식으로도 변환 가능합니다.

TorrentStats

```
#[derive(Serialize, Debug)]
pub struct TorrentStats {
pub state: TorrentStatsState,
pub file_progress: Vec<u64>,
pub error: Option<String>,
pub progress_bytes: u64,
pub uploaded_bytes: u64,
pub total_bytes: u64,
pub finished: bool,
pub live: Option<LiveStats>,
}
```

개별 토렌트의 상세 통계 정보를 제공합니다.

TorrentStatsState 열거형

```rust
#[derive(Clone, Copy, Serialize, Debug)]
pub enum TorrentStatsState {
#[serde(rename = "initializing")]
Initializing,
#[serde(rename = "live")]
Live,
#[serde(rename = "paused")]
Paused,
#[serde(rename = "error")]
Error,
}
```

특징:

- **명시적 이름 지정**: `serde(rename)` 을 사용한 JSON 필드명 제어
- **소문자 표현**: API 일관성을 위한 소문자 상태명

4. 빈 응답 타입

EmptyJsonResponse

```rust
#[derive(Default, Serialize)]
pub struct EmptyJsonResponse {}
```

상태 변경 작업 완료 시 사용되는 빈 JSON 객체 `{}` 를 반환합니다.

JSON 직렬화 패턴

1. 조건부 직렬화

```rust
#[serde(skip_serializing_if = "Option::is_none")]
pub id: Option<usize>,
```

- **메모리 효율성**: 불필요한 null 값 제거
- **API 간소화**: 클라이언트가 처리할 필드 수 감소
- **하위 호환성**: 새 필드 추가 시 기존 클라이언트에 영향 없음

2. 읽기 전용 필드

```rust
#[serde(skip_deserializing)]
pub stats: Option<TorrentStats>,
```

- **데이터 무결성**: 클라이언트가 수정할 수 없는 계산된 값
- **API 안전성**: 잘못된 입력으로 인한 상태 오염 방지

3. 기본값 처리

```rust
#[serde(default)]
pub with_stats: bool,
```

- **사용자 편의성**: 선택적 파라미터의 기본값 자동 적용
- **하위 호환성**: 새 필드 추가 시 기존 요청 호환

4. 커스텀 직렬화 로직

쉼표 구분 목록 처리

```
impl Serialize for OnlyFiles {
fn serialize<S>(&self, serializer: S) -> core::result::Result<S::Ok, S::Error> {
let s = self.0.iter().map(|id| id.to_string()).join(",");
s.serialize(serializer)
}
}
```

- **URL 친화적**: 쿼리 파라미터로 전달하기 쉬운 형식
- **간결성**: 배열 대신 단일 문자열로 표현
- **파싱 효율성**: 클라이언트 측에서 간단한 split 연산으로 처리

오류 처리 및 검증

1. 파싱 오류 처리

```
let list = s.split(',')
.try_fold(Vec::<usize>::new(), |mut acc, c| match c.parse() {
Ok(i) => { acc.push(i); Ok(acc) }
Err(_) => Err(D::Error::custom(format!(
"only_files: failed to parse {:?} as integer", c
))),
```

```
})?;
```

특징:

- **조기 실패**: 첫 번째 오류에서 즉시 중단
- **상세한 오류 정보**: 실패한 값과 기대 타입 명시
- **컨텍스트 제공**: 어떤 필드에서 오류가 발생했는지 표시

2. 비즈니스 로직 검증

```
if list.is_empty() {
return Err(D::Error::custom(
"only_files: should contain at least one file id",
));
}
```

- **의미적 검증**: 구문적으로 올바르지만 의미적으로 잘못된 입력 감지
- **사용자 친화적 메시지**: 기술적 세부사항보다는 사용자 관점의 오류 설명

3. 타입 안전성

```
let tid: TorrentId = v.try_into().map_err(|e| E::custom(format!("{e:#}")))?;
```

- **컴파일 타임 검증**: Rust 타입 시스템을 통한 안전성 보장
- **런타임 검증**: 범위 초과 등의 런타임 오류 처리
- **오류 전파**: 하위 레벨 오류를 상위 레벨로 적절히 전달

API 버전 관리

1. 엔드포인트 버전 관리

```
.route("/torrents/{id}/stats", get(torrents::h_torrent_stats_v0))

.route("/torrents/{id}/stats/v1", get(torrents::h_torrent_stats_v1))
```

- **명시적 버전**: URL 경로에 버전 정보 포함
- **하위 호환성**: 이전 버전 엔드포인트 유지
- **점진적 마이그레이션**: 클라이언트가 자신의 속도로 업그레이드 가능

2. 필드 레벨 호환성

```
#[serde(skip_serializing_if = "Option::is_none")]

pub files: Option<Vec<TorrentDetailsResponseFile>>,
```

- **선택적 필드**: 새 필드를 Option으로 추가하여 호환성 유지
- **조건부 포함**: 필요한 경우에만 추가 정보 제공
- **점진적 확장**: 기존 구조를 깨지 않고 새 기능 추가

3. 기본값 전략

```
#[derive(Serialize, Deserialize, Default)]

pub struct TorrentAddQueryParams {

// ...
```

```
}
```

- **Default 트레이트**: 모든 필드에 합리적인 기본값 제공
- **부분 업데이트**: 일부 필드만 지정해도 동작
- **설정 간소화**: 대부분의 경우 기본값으로 충분

성능 최적화

1. 메모리 효율성

```
#[serde(skip_serializing_if = "Option::is_none")]
```

- **JSON 크기 최소화**: 불필요한 null 값 제거
- **네트워크 대역폭 절약**: 전송 데이터 크기 감소
- **파싱 성능**: 클라이언트 측 JSON 파싱 시간 단축

2. 지연 계산

```
#[serde(skip_serializing_if = "Option::is_none", skip_deserializing)]
pub stats: Option<TorrentStats>,
```

- **조건부 계산**: 요청된 경우에만 통계 계산
- **CPU 절약**: 불필요한 계산 작업 방지
- **응답 시간 개선**: 필수 정보만 빠르게 제공

3. 문자열 최적화

```
let s = self.0.iter().map(|id| id.to_string()).join(",");
```

- **단일 할당**: itertools의 join을 사용한 효율적인 문자열 생성
- **메모리 재사용**: 중간 벡터 생성 없이 직접 문자열 구성

확장성 고려사항

1. 새로운 필드 추가

```
#[derive(Serialize, Deserialize, Default)]
pub struct TorrentAddQueryParams {
// 기존 필드들...
// 새 필드는 Option으로 추가
pub new_feature: Option<bool>,
}
```

- **Option 래핑**: 새 필드를 선택적으로 만들어 호환성 유지
- **기본값 제공**: Default 트레이트를 통한 합리적 기본값
- **점진적 배포**: 클라이언트별로 다른 속도로 새 기능 채택 가능

2. 타입 진화

```
// 기존
pub struct OldType(Vec<usize>);
```

```
// 새로운 버전
pub struct NewType {
pub ids: Vec<usize>,
pub options: Vec<String>,
}
```

- **래퍼 타입**: 내부 구조 변경 시 외부 인터페이스 유지
- **마이그레이션 경로**: 이전 형식에서 새 형식으로의 변환 로직 제공

3. 프로토콜 확장

```
impl TorrentAddQueryParams {
pub fn into_add_torrent_options(self) -> AddTorrentOptions {
AddTorrentOptions {
// 기존 매핑...
peer_opts: Some(PeerConnectionOptions {
connect_timeout: self.peer_connect_timeout.map(Duration::from_secs),
read_write_timeout: self.peer_read_write_timeout.map(Duration::from_secs),
..Default::default()
}),
..Default::default()
}
}
```

```
}
```

- **변환 로직 분리**: API 타입과 내부 타입 간의 명확한 경계
- **기본값 활용**: 새 필드 추가 시 기존 코드 영향 최소화
- **타입 안전성**: 컴파일 타임에 변환 오류 감지

결론

rqbit의 API 타입 시스템은 다음과 같은 특징을 보여줍니다:

1. **강력한 타입 안전성**: Rust의 타입 시스템과 Serde를 활용한 컴파일 타임 검증
2. **유연한 직렬화**: 커스텀 직렬화 로직을 통한 사용자 친화적 API 형식
3. **포괄적인 오류 처리**: 상세하고 이해하기 쉬운 오류 메시지 제공
4. **하위 호환성**: 조건부 직렬화와 기본값을 통한 API 진화 지원
5. **성능 최적화**: 불필요한 데이터 전송 최소화와 지연 계산
6. **확장성**: 새로운 기능 추가 시 기존 클라이언트 호환성 유지

이러한 설계는 BitTorrent 클라이언트의 복잡한 요구사항을 만족시키면서도 개발자 친화적이고 유지보수하기 쉬운 API를 제공합니다.